Inheritance in Java

Inheritance is one of the four fundamental principles of Object-Oriented Programming (OOP), along with **Encapsulation**, **Polymorphism**, and **Abstraction**. It allows a class (child or subclass) to acquire properties and methods from another class (parent or superclass). This promotes code reuse and establishes a hierarchical relationship between classes.

Key Concepts of Inheritance

1. **Parent (Superclass)**:
   - The class whose features are inherited.
   - Also called the **base class**.
2. **Child (Subclass)**:
   - The class that inherits the properties and methods of the parent class.
   - Also called the **derived class**.
3. `extends` **Keyword**:
   - Used to define inheritance in Java.
4. **Single Inheritance**:
   - A class inherits from one superclass.
5. **Multilevel Inheritance**:
   - A class inherits from a class that is itself a subclass.
6. **Hierarchical Inheritance**:
   - Multiple classes inherit from a single superclass.

Why Use Inheritance?

1. **Code Reusability**:
   - Avoids code duplication by reusing parent class methods and properties.
2. **Extensibility**:
   - Enables adding new features to an existing class without modifying it.
3. **Polymorphism**:
   - Allows overriding methods to provide specific implementations in the child class.
4. **Maintainability**:
   - Easier to manage code by centralizing common behavior in a parent class.

Syntax of Inheritance

```
class ParentClass {
    // Properties and methods
}

class ChildClass extends ParentClass {
    // Additional properties and methods of ChildClass
}
```

Example of Inheritance

```
// Superclass (Parent)
class Animal {
    String name;

    void eat() {
        System.out.println(name + " is eating.");
    }
}

// Subclass (Child)
class Dog extends Animal {
    void bark() {
        System.out.println(name + " is barking.");
    }
}

public class InheritanceExample {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.name = "Buddy";  // Inherited from Animal
        dog.eat();           // Inherited method
        dog.bark();          // Method in Dog class
    }
}
```

Output:

```
Buddy is eating.
Buddy is barking.
```

Types of Inheritance in Java

1. **Single Inheritance**:
   - A child class inherits from one parent class.

   Example:

```
class Parent { }
class Child extends Parent { }
```

2. **Multilevel Inheritance**:
   - A class inherits from a class, which in turn inherits from another class.

   Example:

```
class GrandParent { }
class Parent extends GrandParent { }
class Child extends Parent { }
```

3. **Hierarchical Inheritance**:
   - Multiple classes inherit from a single parent class.

   Example:

```
class Parent { }
class Child1 extends Parent { }
class Child2 extends Parent { }
```

`super` Keyword

The `super` keyword is used in inheritance to:

1. Access the parent class's methods or variables.
2. Call the parent class's constructor.

Example:

```
class Animal {
    String name;
```

```
    Animal(String name) {
        this.name = name;
    }

    void display() {
        System.out.println("I am an animal.");
    }
}

class Dog extends Animal {
    Dog(String name) {
        super(name);  // Calling the parent class constructor
    }

    void display() {
        super.display();  // Calling the parent class method
        System.out.println(name + " is a dog.");
    }
}

public class SuperExample {
    public static void main(String[] args) {
        Dog dog = new Dog("Buddy");
        dog.display();
    }
}
```

Output:

```
I am an animal.
Buddy is a dog.
```

Disadvantages of Inheritance

1. **Tight Coupling**:
   - Changes in the parent class may affect child classes.
2. **Increase in Complexity**:
   - Deep inheritance hierarchies can make the code harder to understand and maintain.
3. **Potential for Misuse**:
   - Inappropriate use of inheritance may lead to fragile designs.

Best Practices

1. Use inheritance only when there is a clear **"is-a"** relationship.
    - Example: A Dog "is-a" type of Animal.
2. Use `super` to access parent class members when needed.